



PIMPRI CHINCHWAD EDUCATION TRUST'S.
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV SUD

T.Y. B. TECH

Year: 2025 – 26

Semester: 6

Name of the Student: Sakshi Pandharkar

PRN: 123B1B209

Branch: Computer Engineering

Division: C

Date: 18/2/2026

Assignment 7

Assignment Title:

Design a web application for e-commerce or students feedback review systems or any other applications with MongoDB Backend. using React.js and Express.

Objectives

- To understand backend development using Node.js and to use React.js for frontend development
- To learn routing and middleware using Express.js
- To design a simple e-commerce / Student feedback
- To store and manage data using MongoDB
- To understand client-server communication
- To perform basic CRUD operations

Outcomes

- Develop a functional Student feedback review system
- Implement server-side logic using Node.js and Express and frontend logic using React.js
- Store and retrieve data from MongoDB
- Understand REST API concepts
- Improve full-stack web development skills

Theory: Student Feedback Review System

• Introduction

The Student Feedback Review System is a web-based application developed using the MERN stack. It allows students to submit feedback, which is stored in a database and displayed on the website. The system ensures smooth communication between frontend and backend while managing data efficiently.

- **Technologies Used**

- React.js – for designing user interface
- Node.js – for backend execution
- Express.js – for handling server and APIs
- MongoDB – for storing feedback data
- MongoDB Compass – for viewing and managing database

- **Working of the System**

1. User Input

- The user enters name and feedback in the form on the website (React frontend)

2. Data Transmission

- The data is sent to the backend using HTTP POST request

3. Server Processing

- The backend (Node.js + Express.js) receives the data
- It processes the request and sends it to MongoDB

4. Data Storage

- The data is stored in MongoDB as a document:

```
{  
  "name": "Mrinmayee",  
  "feedback": "Very useful system"  
}
```

5. Data Retrieval

- When the page loads, a GET request is sent
- The backend fetches data from MongoDB
- Data is displayed on the website

6. Database Verification

- The stored data can also be viewed in MongoDB Compass. This confirms that the backend and database are properly connected

- **CRUD Operations Implemented**
 - Create: Add new feedback
 - Read: Display feedback on website
- **Client–Server Communication**
 - React (client) communicates with Express (server)
 - Server interacts with MongoDB
 - Data flows in both directions using REST API
- **Key Features**
 - Simple user interface for feedback submission
 - Real-time data display
 - Database integration with MongoDB
 - Easy data management using MongoDB Compass
- **Advantages**
 - Fast and efficient data handling
 - User-friendly interface
 - Scalable for future improvements
 - Helps in collecting and analyzing feedback

Code:

Server.js

```
const express = require('express');  
const mongoose = require('mongoose');  
const cors = require('cors');  
  
const app = express();  
  
app.use(cors());  
app.use(express.json());
```

```
// connect MongoDB

mongoose.connect('mongodb://127.0.0.1:27017/feedbackDB')

  .then(() => console.log('MongoDB connected'))

  .catch(err => console.log(err));
```

```
// model

const Feedback = mongoose.model('Feedback', {

  name: String,

  message: String

});
```

```
// add feedback

app.post('/feedback', async (req, res) => {

  try {

    const f = await Feedback.create(req.body);

    res.json(f);

  } catch (e) {

    res.status(500).json({ error: e.message });

  }

});
```

```
// get feedback

app.get('/feedback', async (req, res) => {

  const list = await Feedback.find();

  res.json(list);

});
```

```
// start server
app.listen(5000, () => console.log('Server running on port 5000'));
index.html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta
      name="description"
      content="Web site created using create-react-app"
    />
    <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
    <!--
      manifest.json provides metadata used when your web app is installed on a
      user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-
      manifest/
    -->
    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
    <!--
```

Notice the use of %PUBLIC_URL% in the tags above.

It will be replaced with the URL of the `public` folder during the build.

Only files inside the `public` folder can be referenced from the HTML.

Unlike `/favicon.ico` or `favicon.ico`, `%PUBLIC_URL%/favicon.ico` will work correctly both with client-side routing and a non-root public URL.

Learn how to configure a non-root public URL by running `npm run build`.

-->

<title>React App</title>

</head>

<body>

<noscript>You need to enable JavaScript to run this app.</noscript>

<div id="root"></div>

<!--

This HTML file is a template.

If you open it directly in the browser, you will see an empty page.

You can add webfonts, meta tags, or analytics to this file.

The build step will place the bundled scripts into the <body> tag.

To begin the development, run `npm start` or `yarn start`.

To create a production bundle, use `npm run build` or `yarn build`.

-->

</body>

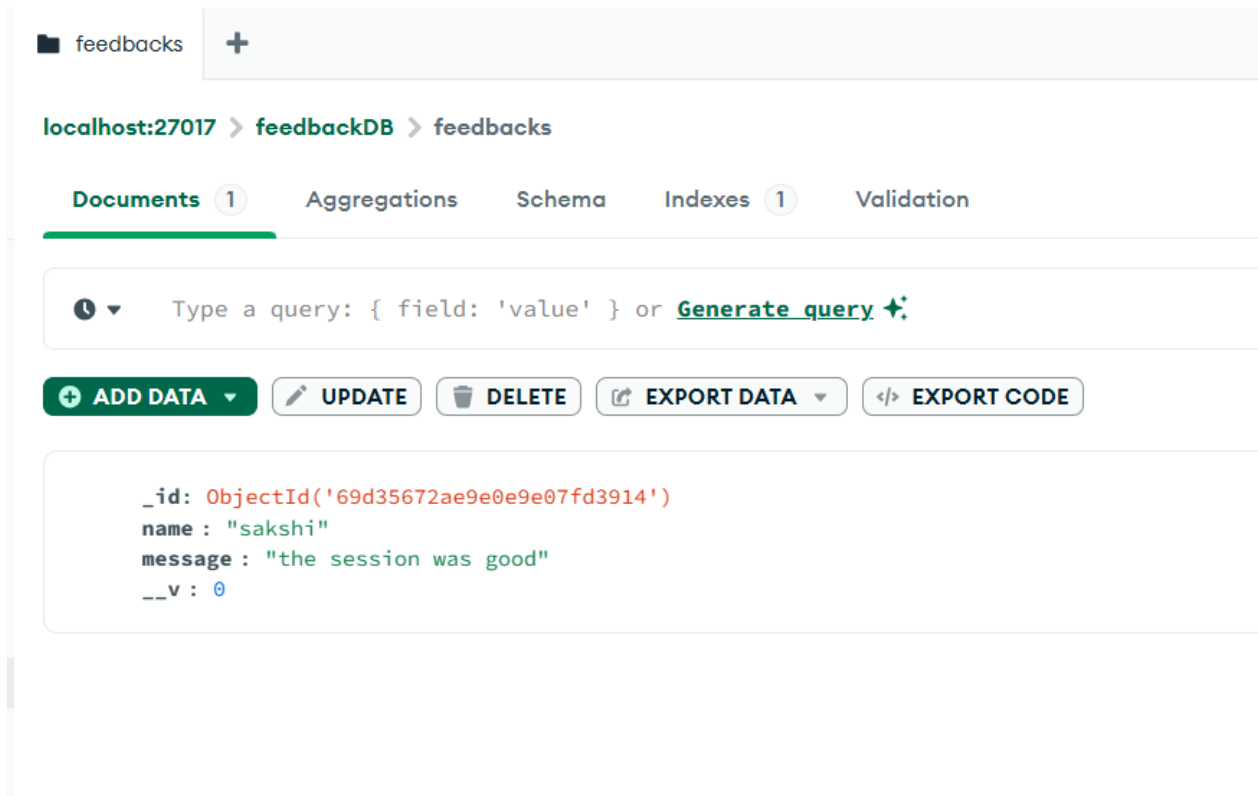
</html>

Student Feedback System

All Feedback

sakshi

the session was good



Conclusion

The Student Feedback Review System successfully demonstrates full-stack development using the MERN stack. The application performs data submission, storage, and retrieval effectively, ensuring proper client-server communication and database management.